Lecture 1.9: Numbers and `Math` in JavaScript

What you covered in the original lecture

This lecture demonstrates common numeric utilities in JavaScript using the built-in `Math` object:

- Constants like `Math.PI` and `Math.E`
- Power and square root: `Math.pow`, `Math.sqrt`
- Rounding: `Math.round`, `Math.floor`, `Math.ceil`
- Formatting decimals with `toFixed`
- Random numbers and generating a number between a range

The `Math` object basics

`Math` is a built-in object that contains useful numeric functions and constants. You do not need to create it; you just use it.

Common constants

```
console.log(Math.PI); // 3.141592653589793
console.log(Math.E);  // 2.718281828459045
```

Output:

```
3.141592653589793
2.718281828459045
```

Insight: These constants are provided at full IEEE double precision; use them in formulas and format for display when needed.

Powered by SPRK Technologies

Power and square root

```
console.log(Math.pow(3, 4)); // 81
console.log(Math.sqrt(385)); // ~19.621...
```

Output:

```
81
19.621416870348583
```

Insight: For exponentiation prefer the `**` operator for readability: `3 ** 4`. `Math.sqrt` returns a floating-point number; format if you need a fixed number of decimals.

> **Modern tip:** You can also use the exponent operator: `3 ** 4` instead of `Math.pow(3, 4)`.

Rounding numbers

Method	What it does	Example
`Math.round(x)`	Nearest integer	`Math.round(3.5) → 4`
`Math.floor(x)`	Round down	`Math.floor(3.999) → 3`
`Math.ceil(x)`	Round up	`Math.ceil(3.001) → 4`

Examples (matching your lecture)

```
console.log(Math.round(435345.35243523));
console.log(Math.floor(3.99999999));
console.log(Math.ceil(3.00000001));
```

Output:

Powered by SPRK Technologies

```
435345
3
4
```

Insight: Rounding behavior is straightforward but watch boundaries (e.g. .5) and display formatting.

Formatting decimals using `toFixed`

`toFixed(n)` formats a number to `n` decimal places.

```
const x = 435345.35243523;
console.log(x.toFixed(2)); // "435345.35"
```

Output:

```
"435345.35"
```

Insight: `toFixed` is useful for presentation, but remember it returns a string; convert back for numeric operations.

> **Important:** `toFixed` returns a **string**, not a number. If you need a number again, convert it using `parseFloat` or `Number`.

```
const formatted = x.toFixed(2);
const asNumber = parseFloat(formatted);
console.log(typeof formatted); // "string"
console.log(typeof asNumber);  // "number"
```

Output:

Powered by SPRK Technologies

```
"string"
"number"
```

Random numbers with `Math.random()`

`Math.random()` returns a decimal number in the range:

`0 ≤ value < 1`

Key patterns

- Random decimal 0..1: `Math.random()`
- Random integer 0..(n-1): `Math.floor(Math.random() * n)`

Generate a random integer between two numbers (inclusive)

Your lecture defines `randomBetween(min, max)`. The clean standard formula is:

```
function randomBetween(min, max) {
    // Inclusive of both min and max
    return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

Output:

```
(no deterministic output — returns an integer in the inclusive range [min, max])
```

Insight: For reproducible tests, use a seeded RNG or mock `Math.random`. Native `Math.random` is not suitable for cryptographic uses.

Powered by SPRK Technologies

> **Why the `+ 1`?** Because `Math.random()` never reaches 1, so multiplying by `(max - min + 1)` ensures `max` can actually occur.

Example usage

```
for (let i = 1; i <= 10; i++) {
    console.log(randomBetween(45, 60));
}
```

Output:

```
example (10 sample outputs):
47
52
59
45
60
48
51
56
46
50
```

Mini task: right triangle (Pythagorean theorem)

Your lecture ends with a task: Ask for two sides of a right triangle and calculate the third side.

If you have the two legs (a and b), find hypotenuse (c)

Formula: `c = sqrt(a² + b²)`

```
const aText = prompt("Enter side a:");
const bText = prompt("Enter side b:");

if (aText !== null && bText !== null) {
    const a = parseFloat(aText);
```


Powered by SPRK Technologies

```
    const b = parseFloat(bText);

    if (isNaN(a) || isNaN(b) || a <= 0 || b <= 0) {
        console.error("Invalid sides");
    } else {
        const c = Math.sqrt((a * a) + (b * b));
        console.log(`Hypotenuse is ${c}`);
        console.log(`Rounded to 2 decimals: ${c.toFixed(2)}`);
    }
}
```

Output:

```
example (if user enters a=3, b=4):
Hypotenuse is 5
Rounded to 2 decimals: 5.00
```

Common mistakes (quick list)

- Forgetting that `prompt` returns a string (convert before math).
- Assuming `toFixed` returns a number (it returns a string).
- Generating random integers with an off-by-one error (missing `+ 1` in the range size).

Practice tasks

1. Write a function `circleArea(r)` and return area using `Math.PI`.
2. Generate 100 random integers between 1 and 6 (dice). Count how many times each number appears.
3. Ask the user for a number and print: rounded, floor, ceil, and toFixed(2).

JS Refined Notes — Lecture 1.9